

Numerical Analysis Assignment 7

nijatabbasov06

November 2024

Table of Contents

1	Problem 7.1	2
2	Problem 7.2	3
2.1	Extra	4

1 Problem 7.1

```

/assignment7/Pr_7_1.py"
evaluating simple 3 degree pol with x = [1, 2, 3, 4] and t = 2.5:
39.625

derivative : 31.75

integral : 63.4375

generating random values:

x vector =
[-7.37054365348087, 1.5327699253267646, -5.523287384497985, -2.489131298592209, 2.316002052803354, 1.8136031803703752, 0.960996332685677
9, 5.2747000891401878, 5.947182011397597, -0.3691738314720361]

n value = 9

```

Figure 1: Problem 7.1 simple interpolation

```

generating random values:

x vector =
[-4.087818227263447, 0.11578501444734002, -7.352764489666628]

n value = 2

t value = 6.589089518905588

resulting value of the pol at t value:
-184.066974414256

derivative : -53.754215458458255

integral : -725.562954495632

```

Figure 2: Problem 7.1 random values interpolations

Evaluating the 3 degree polynomial with the coefficients 1, 2, 3, 4 at 2.5 we get 39.625. Doing the calculations by hand, we can see that this value is indeed correct. In the figure we create the coefficients randomly and at a random value of t we calculate the value of $p(t)$, its derivative and its integral.

2 Problem 7.2

```
x_data: [103 2 3 4 59 8 100 31 72]
y_data: [129 4 9 18 84 94 1 69 151]

(a)
Polynomial parameters (coefficients): [ 1.29000000e+02  1.23762376e+00 -3.76237624e-02 -2.05820582e-02
 3.85210684e-04  1.18012535e-05 -1.60452645e-07  1.43018657e-08
-2.43748507e-10]
Time taken for (a): 0.000041 seconds

(b)
Polynomial parameters (coefficients): [ 1.29000000e+02  1.23762376e+00 -3.76237624e-02 -2.05820582e-02
 3.85210684e-04  1.18012535e-05 -1.60452645e-07  1.43018657e-08
-2.43748507e-10]
Time taken for (b): 0.000018 seconds

(c)
Polynomial parameters (coefficients): [ 1.29000000e+02  1.23762376e+00 -3.76237624e-02 -2.05820582e-02
 3.85210684e-04  1.18012535e-05 -1.60452645e-07  1.43018657e-08
-2.43748507e-10]
Time taken for (c): 0.000052 seconds
```

Figure 3: Problem 7.2 interpolations coefficients

In part a), by solving the lower triangular matrix, we get the values for the coefficients for the interpolant. In part b), by using the following equation we add a new coefficient for the new x and y values:

$$x_{j+1} = \frac{y_{j+1} - p_j(t_{j+1})}{\pi_{j+1}(t_{j+1})}.$$

Figure 4: Equation for the new coefficient

Moreover, in part c, by using the function in part b) I add each of the data inside *x_data* one by one and create a vector consisting of the coefficients of the newton's method.

2.1 Extra

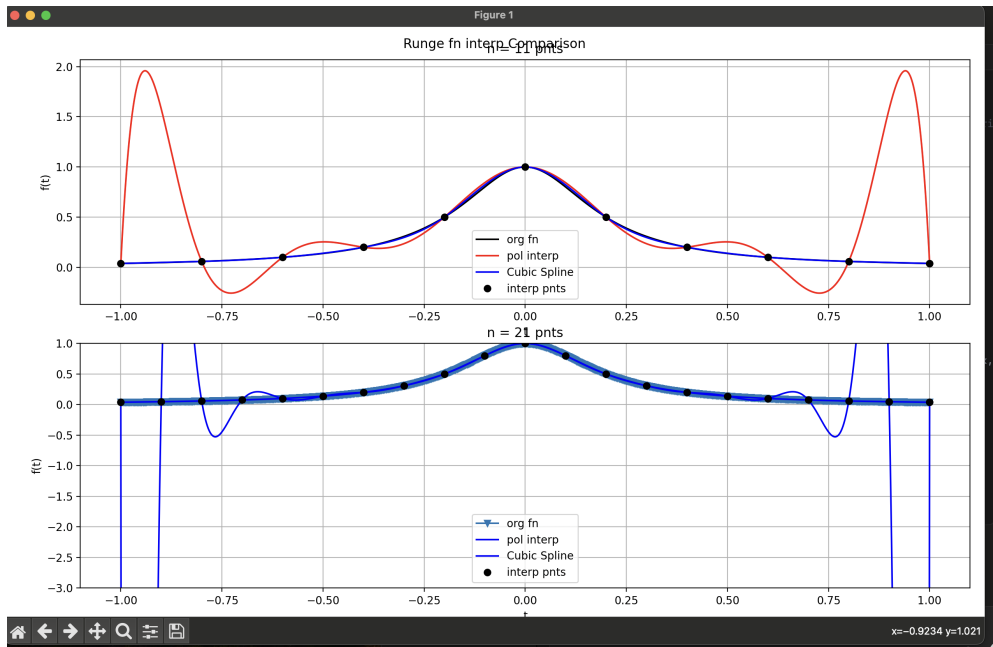


Figure 5: Plots of the curves

```
max abs errs for n=11:
pol interp: 1.915633
Cubic spline interp: 0.021973

max abs errs for n=21:
pol interp: 59.768399
Cubic spline interp: 0.003183
```

Figure 6: Extra Homework Result errors

We can see that when we are interpolating $n = 11$ points, our interpolant is less oscillatory, however, for the case when we interpolated with $n = 22$ points, the interpolant became oscillatory, i.e, it is oscillating a lot between two points because the degree of the polynomial is high and that

is why, the derivative of the polynomial has a lot of roots. For this reason, as the number of interpolating points increase, the interpolant will have more maxima and minima.

Moreover, we can see that the Cubic Spline function converges to the original function better than the polynomial interpolation. The reason for this is that, when we are doing Cubic Spline, we do not have those issues of having a lot of roots for the derivative of the polynomial. For this case, between the interpolating points, the spline does not increase or decrease a lot like it did for the polynomial interpolation.